IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 7 February 2026, accepted 28 February 2026, date of publication 10 March 2026, date of current version 23 March 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3672690

RESEARCH ARTICLE

XGBoost-Based URL Phishing Detection Method With Cross-Dataset Validation

MILOSZ MISIEK AND TOMASZ HYLA
West Pomeranian University of Technology, 71-210 Szczecin, Poland
Corresponding author: Milosz Misiek (milosz.misiek@zut.edu.pl)

ABSTRACT This article analyses the performance characteristics of XGBoost models across multiple datasets for phishing URL detection, extending our previous conference paper with comprehensive cross-dataset validation and comparative analysis. Using a validation framework with three distinct datasets — a custom phishing dataset (75,738 samples), the large-scale GramBeddings dataset (639,723 samples), and the PhiUSIIL dataset (47,103 samples) — we demonstrate that XGBoost delivers robust performance across diverse data sources. Our approach addresses the issue of feature instability, showing how balanced feature engineering is crucial for reliable detection. The model achieves 90.7% accuracy and 91.2% F1 score on the custom dataset, with a solid 77.8% average accuracy across three independent test sets. In comparative benchmarks against Neural Networks, XGBoost proved superior in training efficiency and detection quality, achieving 2.1% higher accuracy and training 12.6 times faster (0.95s vs 12.01s) with 3.6 times less memory. Although Neural Networks offered faster inference (7.51ms vs 20.6ms) and smaller model sizes, XGBoost's balance of high accuracy and rapid training makes it highly effective for practical, real-world phishing detection systems.

INDEX TERMS Cybersecurity, machine learning, phishing detection, XGBoost.

I. INTRODUCTION

Instead of breaking complex security measures, it is much easier to mislead users into providing their private data, sending money to a criminal, or approving an incorrect transfer. The effects of a successful phishing attack can include financial loss, reputational damage, and an increased risk of identity theft. Phishing constitutes a serious threat to information security, emphasising the need to implement effective counteracting strategies and increasing risk awareness among network users. One of the basic steps cybercriminals will take is to prepare false URL (Uniform Resource Locator) addresses that lead to various phishing sites or infect the victim's device with malware. Malicious URLs are sent via short messages or email and posted on various websites, including social networking sites. The phishing attack process consists of several phases, including attack preparation, attack execution, and attack results exploitation [1].

The associate editor coordinating the review of this manuscript and approving it for publication was Sangsoon Lim.

Malicious URLs can be divided into several types [2]: spam URL attacks, phishing URL attacks, malware URL attacks, and defacement URL attacks. Malicious URLs can be classified into multiple classes, but most proposed solutions allow binary classification into only two categories (benign, malicious).

Protecting users from malicious URLs requires continuously monitoring all displayed URLs and alerting users when suspicious links are detected. This protection mechanism can be implemented as an add-on to existing applications. However, designing such plug-ins presents several fundamental challenges. The first challenge involves performance optimisation, particularly when using advanced machine learning models, which may require offloading some computations to a server. The second challenge addresses the detection of previously unseen malicious URLs, which necessitates the use of machine learning or large language models. The third challenge concerns cross-dataset validation, as models trained on one dataset may not generalise well to different URL sources, requiring robust feature engineering and validation across diverse

datasets to ensure reliable detection performance. Based on a preliminary literature review, the XGBoost algorithm was selected due to its proven high effectiveness in classification tasks, its ability to handle imbalanced datasets and superior performance with tabular data. XGBoost is generally faster than many neural networks, especially deeper ones. It naturally handles missing values and provides clear feature-importance scores, which help explain which signals matter most for phishing detection. Because phishing datasets are often imbalanced and high-dimensional, their boosting process tends to produce robust, reliable results by paying extra attention to hard-to-classify cases. Evidence supports this. A study by Hajara et al. [3] reports 97.27% accuracy for XGBoost, outperforming probabilistic neural networks. Furthermore, study by Kumaraswamy and Kumar [4] shows 90.4% accuracy - slightly higher than multilayer perceptrons - along with quicker training and prediction. These strengths make XGBoost a practical option for real-time cybersecurity.

A. RELATED WORK

Prior work on phishing detection can be organised into: URL-only methods, hybrid/multi-modal approaches, LLM-based systems, and domain-adaptation research addressing cross-dataset generalisation.

1) URL-ONLY METHODS

Several studies focus exclusively on URL-based features and apply classical machine learning models to these inputs. For example, Zaidan and Alkasassbeh [5] and Pande and Dhage [6] provide broad reviews of detectors that use features or heuristics derived from URLs. Tan et al. [7] and Asif et al. [8] summarise comparative evaluations of classifiers restricted to URL-only features and discuss typical feature sets. Solutions like Bozkir et al.'s [9] GramBeddings and other URL-centric approaches typically report strong performance on within-dataset splits but do not generally test across datasets, limiting insight into their generalisation.

This focus on classical machine learning models demonstrates the breadth of approaches within URL-only research. To further illustrate the versatility of such models in security applications, Alzubi [10] shows that XGBoost can be effective for cybersecurity classification in a different domain by detecting IoT botnet attacks with metaheuristic hyperparameter tuning, supporting the broader applicability of boosted tree models in security settings.

2) HYBRID AND MULTI-MODAL APPROACHES

Building upon these URL-only methods, hybrid approaches combine information from URLs with additional data sources, such as HTML content, page renderings, screenshots, or NLP-extracted features, to improve detection robustness. For instance, Marchal et al. [11] survey methods that leverage multiple modalities, while Özcan et al. [12] propose deep hybrid models that integrate URL character embeddings with NLP-based features to improve performance across varied settings.

3) LLM-BASED APPROACHES

Alongside these hybrid techniques, more recent developments include the exploration of large language models and ensemble LLM strategies for phishing detection. For example, Nasser et al. [13] investigate stacking and ensembling multiple LLMs, achieving strong reported accuracy at the cost of substantially higher computational requirements. As with other LLM applications, these approaches trade performance for resource intensity and deployment complexity.

4) DOMAIN-ADAPTATION AND CROSS-DATASET RESEARCH

Rounding out this landscape, work specifically addressing cross-dataset generalisation and domain shift is emerging. Rashid et al. [14] propose an Unsupervised Domain Adaptation (UDA) framework to align source and target feature spaces without requiring target labels, demonstrating that state-of-the-art models can suffer 10%–32% F1 drops under dataset shift. Akçam et al. [15], and others, highlight that training and evaluating on separate datasets is necessary to reveal generalisation limitations that are often masked by single-dataset evaluations.

B. CONTRIBUTION

This paper introduces an XGBoost-based machine learning method for detecting malicious web addresses in phishing attacks. The model achieves strong cross-dataset generalisation, computational efficiency for real-time use, and reveals connections between feature importance stability and predictive performance.

Compared to our earlier conference paper [16], this extended study adds a comprehensive introduction, a generalised model evaluated across datasets, extra experiments, thorough performance evaluations, and deeper methodological insights.

We address three key research questions:

1) Can XGBoost achieve statistically significant cross-dataset generalisation compared to baseline methods?
2) What is the relationship between feature importance stability and cross-dataset performance?
3) How do computational efficiency characteristics compare between XGBoost and neural networks for real-time deployment?

Table 1 summarises the concrete extensions made in this journal submission compared to the earlier conference paper. It highlights the expanded datasets and the move from single-dataset evaluation to cross-dataset validation. The table also notes the adoption of rigorous cross-dataset feature-selection criteria, the use of Bayesian hyperparameter optimisation, and more detailed per-dataset results and thresholds.

Our optimised XGBoost model achieves an average F1 score of 77.93% across datasets (GramBeddings: 73.25%, Custom: 91.16%, PhiUSIIL: 69.37%), demonstrating robust detection performance. The model balances recall and specificity, reducing false outcomes, and delivers real-time prediction throughput suitable for browser-based security through efficient features and tuning.

TABLE 1. Comparison with the earlier conference version.

Aspect	Conference version	This extended journal version
Datasets	Single dataset evaluation	Cross-dataset evaluation (Custom, GramBeddings, PhiUSIIL) with held-out tests
Experiments	Basic train/test split on one dataset	Extensive cross-dataset validation, threshold analysis, and computational benchmarking
Feature selection	Within-dataset selection	Rigorous cross-dataset criteria and final set of 8 stable features
Hyper-parameters	Simple tuning	Bayesian optimisation and expert-guided adjustments; reported parameters included
Results	High single-dataset accuracy	Detailed cross-dataset metrics (average F1 77.93%), and per-dataset breakdowns

II. RESEARCH METHOD

This research aims to develop a phishing detection system using a machine learning algorithm, implemented as a browser extension for real-time protection. The goal is to develop an XGBoost-based phishing detection method (X-PHIDE) that takes a URL address as input and returns a phishing prediction.

The method uses a machine learning model, a simple allowlist of safe domains, and a blocklist of unsafe domains. These resources are integrated into a browser plug-in that collects website addresses and visually classifies them into three categories: *potentially harmful*, *suspicious*, or *safe*.

The URL tagging algorithm is activated when a user clicks on an external link. It first checks whether the domain appears on the allowlist or blocklist. If the domain is not found on either list, the plug-in consults the machine learning model, which provides a probability that it is a phishing attempt.

If the domain is on the allowlist or the model estimates the phishing probability at less than 40%, it is marked as *safe*. If the domain is on the blocklist or the phishing probability exceeds 80%, it is flagged as *potentially harmful*. Domains between 40% and 80% phishing probabilities are labelled as *suspicious*.

Users can also manually confirm the classification of any domain, and these decisions are saved to the appropriate allowlist or blocklist for future reference.

In the designed system, the URL is captured as input, followed by feature extraction to derive relevant characteristics. These features serve as inputs to the machine learning model, which predicts whether the URL is phishing or benign. The Python-based system is presented in Figure 1.

Initially, the model was trained using the default hyperparameters provided by the XGBoost library. The default parameters of the XGBoost classifier used as a baseline are summarised in Table 4. These settings provided a starting point for training before hyperparameter optimisation, described in Section II-C.

A. DATASETS

The model was trained on a custom dataset prepared from four data sources:

1) PhishTank — providing the latest phishing URLs with data downloaded in March 2024 [17].
2) AdaURL — a collection of legitimate URLs released on March 2, 2023 [18].
3) The Kaggle Phishing URLs Dataset by Hassaan Mustafavi, which was published on January 25, 2025, containing over 45,000 URLs labelled as phishing or legitimate [19].
4) PhishDataset — a balanced dataset, available on GitHub and collected from May 2021 to June 2021 [20].

This combination of datasets provides a balanced, up-to-date representation of phishing and legitimate URLs, enabling robust training and evaluation of phishing-detection models. The final dataset combined randomly selected URLs from the above sources.

Next, a comparative feature analysis was conducted on the GramBeddings dataset [21] — a freely available dataset of phishing and legitimate URLs — to identify the most stable predictors. Cross-dataset feature intersection was performed to select features that remain discriminative across both data sources. Eight of the ninety available features were finally used to train the model.

A third dataset, the PhiUSIIL dataset comprising 134,850 legitimate and 100,945 phishing URLs [22], was primarily used for testing (utilising a fully feature-compatible subset of 47,103 URLs). During the testing phase, we discovered that the HTTPS protocol was a strong contributor to correct classification in PhiUSIIL, but a weak contributor in the GramBeddings test dataset. Details are presented in Section III.

The selection of these datasets was motivated by their scale and diversity, both of which are essential for training robust phishing detection models. The custom dataset aggregates multiple sources to capture a broad spectrum of phishing techniques and legitimate URL structures, thereby improving the model's generalisation. The GramBeddings dataset serves as a large-scale benchmark for feature importance analysis. Its phishing URLs were collected from real-world feeds such as PhishTank and OpenPhish over an extended period (May 2019–June 2021), thereby reducing temporal bias. In addition, similarity reduction techniques were applied to ensure that the samples are representative rather than repetitive. The dataset was specifically designed to support character-level and n-gram-based learning, offering substantial variability in URL structures.

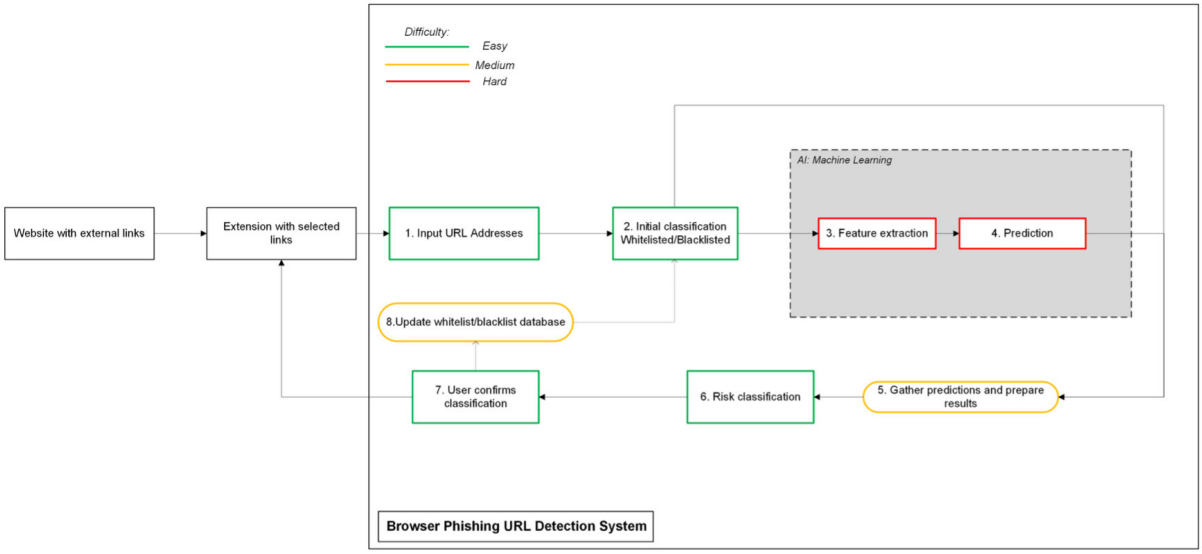


FIGURE 1. System overview of the X-PHIDE phishing detection method.

The PhiUSIIL dataset is used as an independent test set to assess cross-dataset generalisation. It represents a realistically sized collection that includes both common and more sophisticated phishing instances, and it reflects recent, real-world URLs—an important property given the rapidly evolving nature of phishing attacks.

To quantify differences between the datasets used in this study, Table 5 reports simple per-dataset summary statistics (sample size, phishing prevalence and selected feature summaries). Figure 2 and Figure 3 visualise two of the most informative distributional differences: HTTPS adoption rates and URL length distributions. These distributional contrasts help explain why some features (for example, `HTTPS` or URL length) display markedly different importances across datasets and why cross-dataset performance can vary.

Overall, this multi-dataset evaluation strategy mitigates overfitting to a single data source and improves performance in real-world deployment scenarios.

To control for potential URL leakage between datasets, we measured cross-set overlap using four increasingly coarse keys: the raw URL string, a canonicalised URL with sorted query parameters, a canonicalised URL without query parameters, and the eTLD+1 domain (best-effort). Exact overlaps between the independent datasets are small, while the largest overlap is observed between the GramBeddings training set and its provided test split. Full overlap statistics are reported in Table 6.

B. FEATURE EXTRACTION AND REDUCTION

The model was trained on features extracted from a custom Python system. The feature extraction method is presented in Figure 4.

This method follows a multi-stage approach, involving the extraction of over 90 features from URLs and rigorous cross-dataset validation to identify the most reliable indicators.

The extracted features included lexical features, DNS and network features, security and authentication features, domain reputation, and redirect and URL behaviour. Since each dataset contained a different set of extracted features, common ones were selected and analysed to identify the best set for cross-dataset optimization. This involved analysing the importance of each feature in each dataset, followed by evaluating its importance across all datasets to select the most reliable features. The selection criteria are presented in Table 2.

TABLE 2. Feature selection criteria.

Criterion	Threshold	Purpose
Mean Importance	> 0.02	Significant predictive power
Minimum Importance	> 0.005	Always positive contribution
CV (Stability)	< 0.8	Consistent across datasets
Dataset Presence	≥ 0.75	Available across most datasets

The feature selection criteria were developed through rigorous, data-driven analysis, prioritising reliability over raw performance. The four criteria work as a comprehensive evaluation framework:

- **Mean Importance > 0.02:** Filters for predictive power, representing 2% of total model importance, which is significant enough to contribute meaningfully without dominating decisions.
- **Minimum Importance > 0.005:** Ensures features contribute positively even in worst-case scenarios, preventing degradation of performance across datasets.
- **CV (Stability) < 0.8:** Measures consistency using the coefficient of variation (CV), allowing for natural dataset variations while ensuring feature reliability.

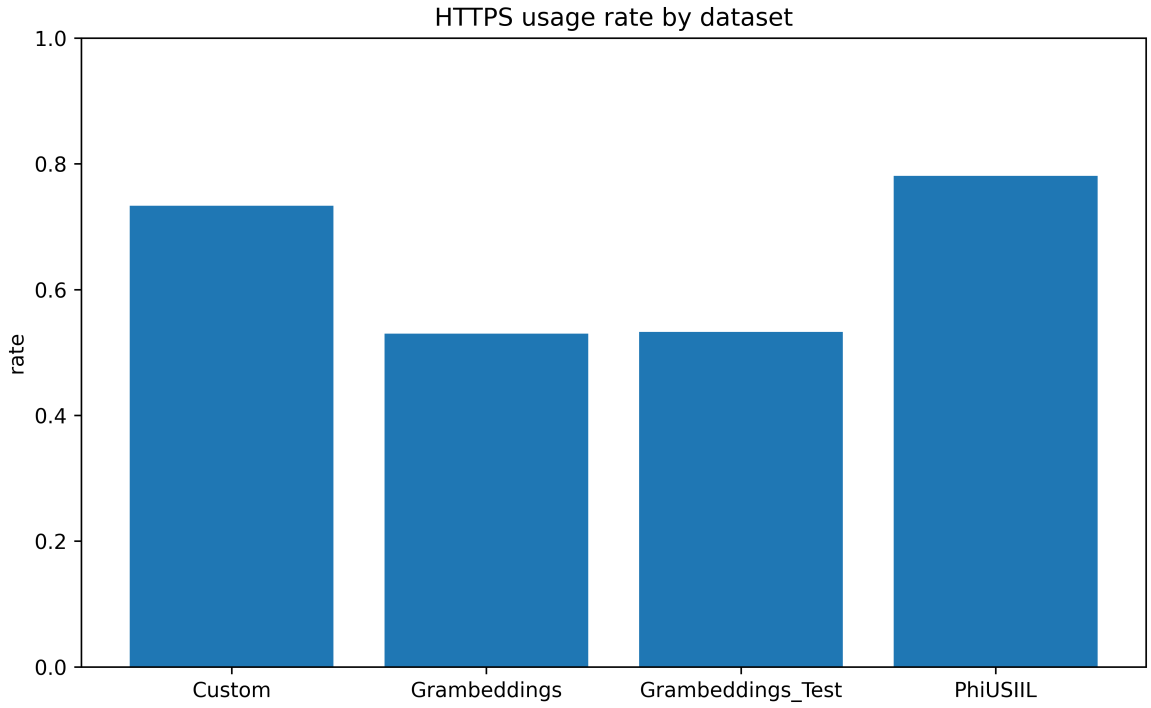


FIGURE 2. HTTPS usage rate by dataset.

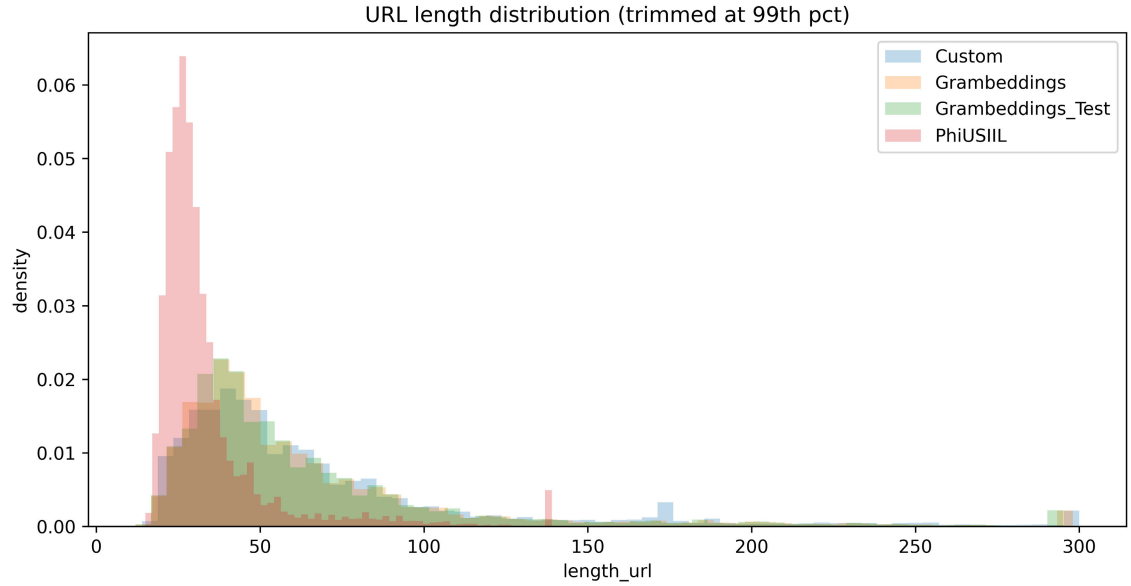


FIGURE 3. URL length distribution across datasets (trimmed at 99th percentile).

- **Dataset Presence $\geq$ 0.75:** Requires features to be available across at least 75% of datasets, preventing overfitting and ensuring practical applicability in diverse environments.

The stringent nature of these criteria is evident — only one feature (`length_url`) met all requirements out of 32 analysed features. This demonstrates the rigour of the selection process and the challenge of finding reliable features for cross-dataset phishing detection. The criteria reflect a balanced approach valuing consistency and generalisation over raw performance, ensuring effective deployment across different environments.

The final model uses eight features selected for cross-dataset balanced performance. Together, these features provide effective classification of phishing and benign URLs with strong cross-dataset generalization. Table 7 presents the final selected features, their type, importance, stability, and the reason for selecting them.

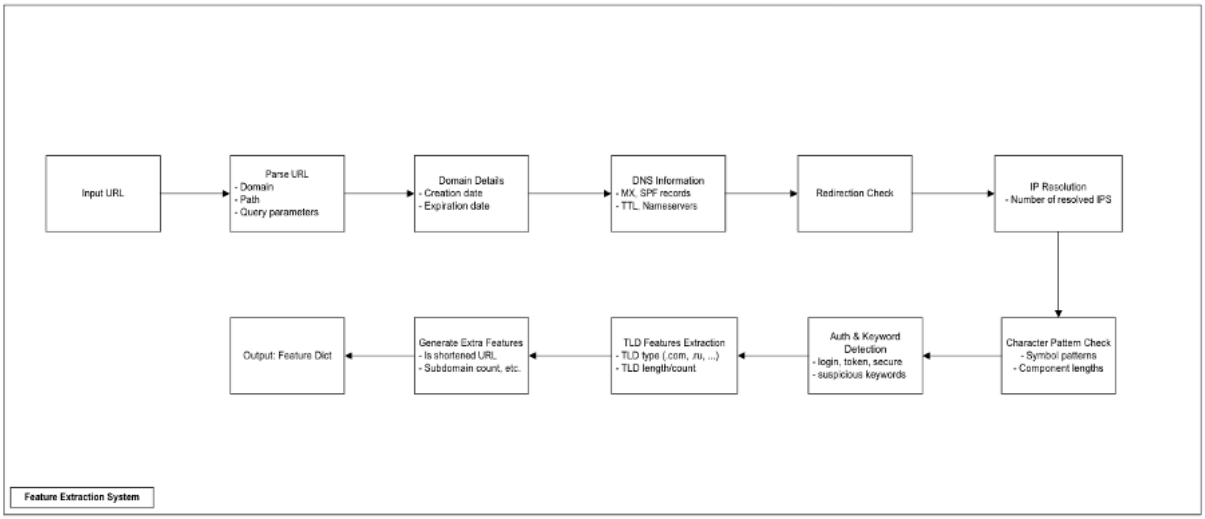


FIGURE 4. Feature extraction method.

C. HYPERPARAMETER TUNING

We tuned XGBoost hyperparameters using Bayesian optimisation to balance model capacity and computational efficiency for phishing URL detection. The search was conducted with `BayesSearchCV` from Python's `scikit-optimize` library on the custom dataset using 5-fold stratified cross-validation and a budget of 100 evaluations, parallelised across all available cores (`n_jobs = -1`). The optimisation objective was the estimator's default cross-validated accuracy. The initial search space is presented in Table 8. These ranges were selected to ensure sufficient expressiveness while constraining overfitting and runtime.

After optimisation, we refit the best configuration on the training split and evaluated generalisation on three test sets: the GramBeddings test set, a held-out split from the custom dataset, and the PhiUSIIL test set when available. To align performance with operational requirements, we reported metrics at a fixed decision threshold of 0.85 and optimised thresholds via ROC analysis to meet target specificity levels (0.90–0.95). We also report feature importance rankings from the selected model to aid interpretability.

Following Bayesian optimisation, we performed limited expert-guided adjustments outside the search loop to explore cost-sensitive configurations and simple ensembles, trading recall and specificity at application-relevant operating points. In particular, class imbalance was handled using XGBoost's native `scale_pos_weight`, which we tuned to approx. 1.95. We chose this approach because it integrates directly into the boosting objective, is computationally lightweight, and preserves the original URL data distribution—avoiding the potential distortions introduced by synthetic resampling of lexical URL patterns. While alternatives such as SMOTE or custom objectives (e.g., focal loss) can be effective in some contexts, they introduce additional generation or implementation complexity and may require substantially more tuning when applied with tree boosters. For these reasons, `scale_pos_weight` provided a simple, interpretable, and effective remedy for imbalance in our pipeline. Table 9 summarises selected hyperparameters with initial values comparison and engineering reasons.

III. RESULTS

All experiments and model training were conducted on a workstation equipped with the following hardware specifications: Processor (CPU): Apple M1 Pro 8-core CPU with six performance cores and two efficiency cores; Memory (RAM): 16 GB LPDDR5 RAM; Graphics Processing Unit (GPU): Apple M1 Pro with 14 built-in cores; Storage: 512 GB SSD; Operating System: macOS Sequoia 15.5.

The development environment included Python 3.9 with key libraries such as `XGBoost` for model training, `scikit-learn` for preprocessing and evaluation, and JavaScript for browser extension implementation. For API development, the `FastAPI` Python microframework was used. For domain data and blacklists/whitelists, a NoSQL database, `MongoDB`, was used.

Table 3 summarises the performance metrics of the optimised `XGBoost` model evaluated on the custom phishing detection dataset. The model achieved an excellent accuracy of 90.7% on the test data. The sensitivity (recall) indicates that 94.6% of phishing URLs were correctly detected. The specificity shows that 86.8% of legitimate URLs were correctly classified as non-phishing, demonstrating good precision in avoiding false alarms. The precision confirms that 88.0% of URLs flagged as phishing were malicious, indicating high confidence in positive predictions. The F1 score of 91.2% combines precision and recall into a single comprehensive metric, reflecting the model's balanced performance.

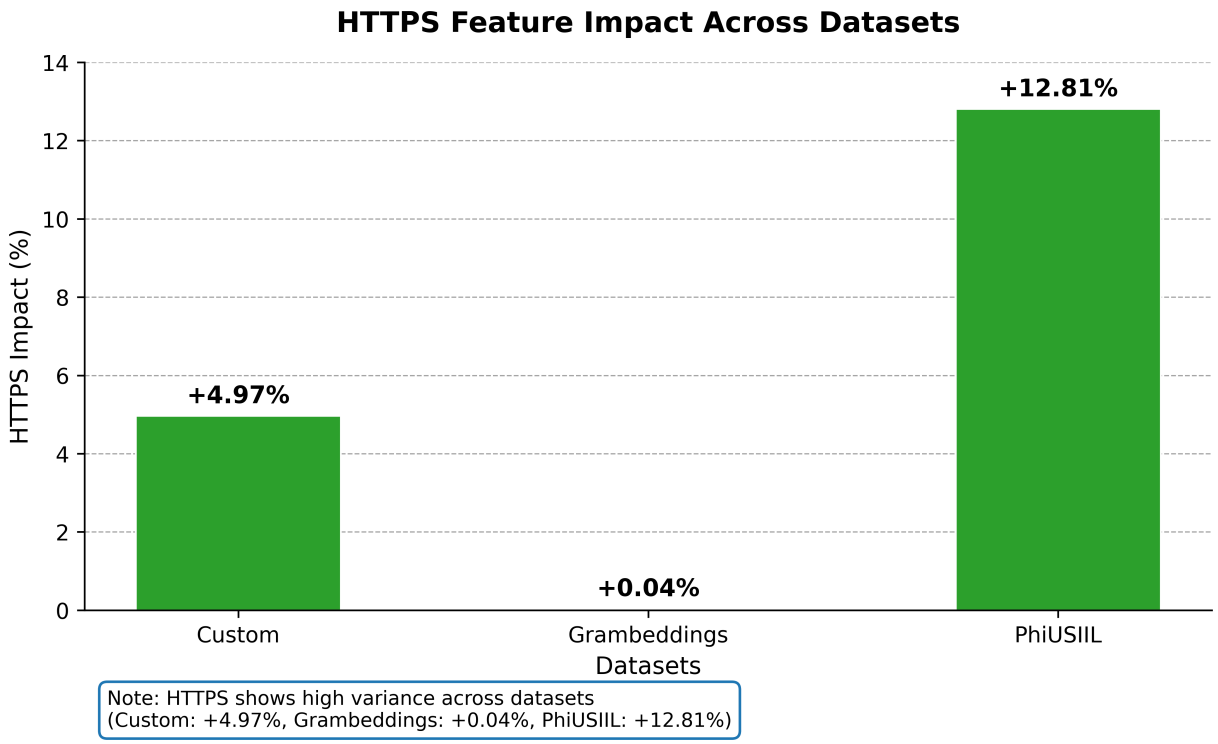


FIGURE 5. Impact of the `HTTPS` feature across the three datasets.

TABLE 3. Performance metrics of the optimised XGBoost model on the custom phishing detection dataset.

Metric	Value (%)
Accuracy	90.7
Sensitivity (Recall)	94.6
Specificity	86.8
Precision	88.0
F1 Score	91.2

A. FEATURE IMPORTANCE ANALYSIS ACROSS DATASETS

To understand how the model generalises between datasets, the feature importance was evaluated across three distinct datasets:

1) Custom dataset (75,738 URLs),
2) GramBeddings dataset (639,723 URLs),
3) PhiUSIIL dataset (47,103 URLs).

The `HTTPS` feature shows the greatest variation in discriminative power, ranging from 0.125 in the GramBeddings dataset to 0.513 in the PhiUSIIL dataset. This directly correlates with model performance improvements: an increase of +0.04% for GramBeddings versus +12.81% for PhiUSIIL.

The cause of this discrepancy lies in dataset-specific URL characteristics. When evaluating the GramBeddings dataset, both phishing and legitimate URLs exhibit similar HTTPS adoption patterns (46.8% for phishing versus 59.2% for legitimate). This results in limited discriminative value.

In contrast, the PhiUSIIL dataset shows a dramatic difference: 100% of legitimate URLs use HTTPS compared to only 48.7% of phishing URLs, creating a strong discriminative signal for the model.

Figure 5 illustrates the impact of the `HTTPS` feature on the three datasets included in the study.

To assess reliance on high-impact features, we include a targeted ablation-style check for `HTTPS`: Figure 5 reports the change in accuracy when the feature is included versus excluded. The effect is strongly dataset-dependent (GramBeddings: +0.04% accuracy; PhiUSIIL: +12.81%), indicating the model does not rely on a single shortcut uniformly across data sources. For URL length, we report dataset-specific length distributions visualised in Figure 3.

Despite the variations, eight of the attributes have a positive contribution across all datasets. These features consistently demonstrate their relevance in phishing detection, regardless of dataset-specific characteristics. Table 10 lists these features and their corresponding calculation methods.

These attributes maintain their discriminative power across datasets, suggesting they can capture intrinsic patterns in URL behaviour and structure.

The `length_url` characteristic shows particular robustness, with a mean importance of 0.0319 and moderate stability (CV: 0.7843) across datasets. This uniformity indicates that URL length patterns are a phishing feature that persists across all datasets, although the magnitude of the effect remains dataset-specific.

B. CROSS-DATASET GENERALIZATION RESULTS

Table 11 presents model performance on three datasets that test the model's generalisation ability. To select operating points, we swept a small set of candidate thresholds (0.3,

0.4, 0.5, 0.6, 0.7, 0.8, 0.85, 0.9) and selected the threshold that maximised a compact custom objective combining recall, F1 and specificity. The custom objective gives higher weight to specificity when specificity is low (this reflects a practical deployment constraint: excessive false positives are costly), otherwise it balances recall and F1. This threshold-search procedure produces two useful reporting points: a standard balanced threshold (0.50) that characterises in-distribution performance (F1 91.16%, specificity 86.60%), and a conservative, deployment-oriented threshold (0.85) that was favourable for cross-dataset evaluations because it increases operational specificity at the expense of some recall (GramBeddings: F1 73.25%, recall 80.25%, specificity 60.94%; PhiUSIIL: F1 69.37%, recall 73.76%, specificity 70.97%).

In short, thresholds reported in Table 11 were chosen by a deliberate threshold-sweep and custom-scoring rule. The 0.50 represents the balanced in-distribution operating point, while 0.85 illustrates a conservative, specificity-focused operating point appropriate for unseen datasets and practical deployment scenarios. These choices reflect operating trade-offs rather than arbitrary numeric cutoffs.

To address the baseline-comparison requirement, we additionally benchmarked XGBoost against Logistic Regression, Random Forests, LightGBM, CatBoost, and a simple MLP using the same feature set and evaluation splits. For a fair, deployment-aligned comparison, we report metrics at a fixed decision threshold of 0.50 for all models in Table 12. Across datasets, boosted tree models (XGBoost/LightGBM) deliver the strongest overall balance of F1 and ROC-AUC while maintaining high recall, whereas linear baselines degrade substantially under cross-dataset shift. We therefore select XGBoost as the primary model due to its consistently strong cross-dataset performance, fast training/inference on tabular URL features, and built-in interpretability via feature-importance scores.

C. PERFORMANCE TEST RESULTS

XGBoost's complexity grows linearly, while neural networks' complexity scales with the product of the numbers of neurons in adjacent layers, which can be very large for deep or wide networks, making the training complexity much higher. XGBoost typically requires fewer boosting rounds (trees) to converge on structured data, while neural networks often require many epochs to train effectively. XGBoost's algorithm is highly optimised for parallel processing trees and columns, speeding up training.

As a result, XGBoost is generally faster to train on structured, sparse datasets, whereas neural networks are preferred for unstructured data (e.g., images, text) despite higher computational demands.

Despite some advantages of deep learning approaches, XGBoost shows better overall performance than neural networks for phishing URL detection. Neural networks have faster prediction times (7.51 ms versus 20.59 ms) and smaller model sizes (0.18 MB versus 2.37 MB) due to their simple matrix operations and compact parameter storage. However, XGBoost provides superior results that outweigh these benefits.

Training speed is significantly better — XGBoost trains 12.6 times faster than neural networks (0.95 s versus 12.01 s), making it much more practical for model development and tuning. Prediction quality is notably higher, with XGBoost achieving 2.1% higher accuracy (90.7% versus 88.7%) and 2.8% higher F1 score (91.2% versus 88.7%), which is crucial for security applications where missing phishing URLs is costly.

Deployment is much simpler — XGBoost requires no GPU acceleration, deep learning frameworks, or complex preprocessing. The model works directly with raw features without standardisation, making it ideal for production environments where resources and complexity are concerns.

The neural network's speed and size advantages come from its simpler computational approach — it performs matrix multiplications rather than tree traversal, and stores only weights rather than complete tree structures. However, these advantages are outweighed by XGBoost's better accuracy, faster training, and easier deployment, making it the optimal choice for phishing detection systems where performance quality matters most.

D. PLUG-IN TESTING

The classification model was implemented in a plug-in application for the Google Chrome browser. The XGBoost library was used, which allows the model to be easily loaded and used in the application. The classification model was saved in `.pkl` format using the `pickle` library. When the plug-in is launched, the application loads the model and is ready to classify URLs. Figure 6 presents a user interface displaying the plug-in activated in the browser with selected URLs and their classification.

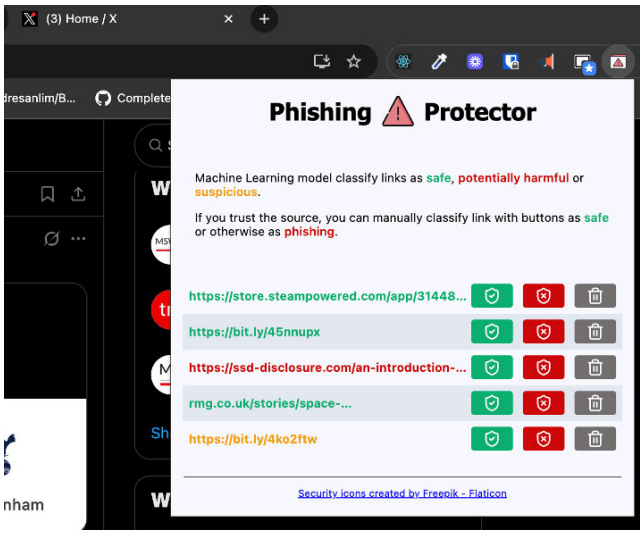


FIGURE 6. Browser plug-in with latest classification model.

TABLE 4. XGBoost model's default parameters.

Parameter	Default Value	Description
booster	gbtree	Booster type: `gbtree`, `gblinear`, or `dart`
n_estimators	100	Number of boosting rounds
max_depth	6	Max tree depth
eta	0.3	Learning rate
gamma	0	Minimum loss reduction
subsample	1	Subsample ratio
colsample_bytree	1	Column subsample ratio
lambda	1	L2 regularization
alpha	0	L1 regularization

TABLE 5. Datasets summary statistics.

Dataset	Rows	Phishing Rate	URL Length Median	HTTPS Rate	Domain Age Median	Nameservers Mean	SPF Rate
Custom	75,738	0.5057	52	0.7333	12,395.01	2.97	0.8054
Grambeddings	639,723	0.5001	49	0.5301	11,018.94	0.89	0.9600
Grambeddings Test	159,940	0.5002	49	0.5326	10,584.10	0.85	0.9610
PhiUSIIL	47,103	0.4274	28	0.7808	19,643.31	1.97	0.3496

Domain Age is reported as the median of `time_domain_activation`/86400 (days).

The API for URL classification has been upgraded to utilise Docker containerisation. The new API is not deployed to the cloud provider but can be used locally. This approach was tested, and the result is shown in Figure 7.

Our study introduces a more rigorous evaluation framework by assessing model performance across multiple independent datasets, addressing a critical limitation in previous phishing detection research. While our previous single-dataset evaluation achieved 97.80% accuracy, the cross-dataset analysis reveals a more realistic average accuracy of 77.84% across three diverse test datasets (Custom Test: 90.72%, GramBeddings Test: 70.65%, PhiUSIIL Test: 72.16%). This performance difference (19.96 percentage points) highlights the impact of dataset shift and the challenge of model generalisation across different data distributions and collection methodologies, a factor often overlooked in existing literature.

The cross-dataset evaluation demonstrates that our model maintains reasonable performance, with an average recall of 82.89% and an F1 score of 77.93%, indicating robust phishing detection capabilities even when tested on unseen data distributions. However, the 73.60% average precision and 72.90% specificity suggest room for improvement in reducing false positives across diverse datasets. This comprehensive evaluation provides a more realistic assessment of real-world deployment scenarios, where models must generalise beyond their training data distribution.

The performance variation across datasets (ranging from 70.65% to 90.72% accuracy) underscores the importance of dataset diversity in model evaluation and the need for robust feature engineering that can adapt to different URL characteristics and phishing patterns. This cross-dataset analysis represents a more stringent and realistic evaluation than single-dataset studies, providing valuable insights for the practical deployment of phishing detection systems. The results are presented in Table 13. It is important to note that the compared models use only the train/test split on a single dataset.

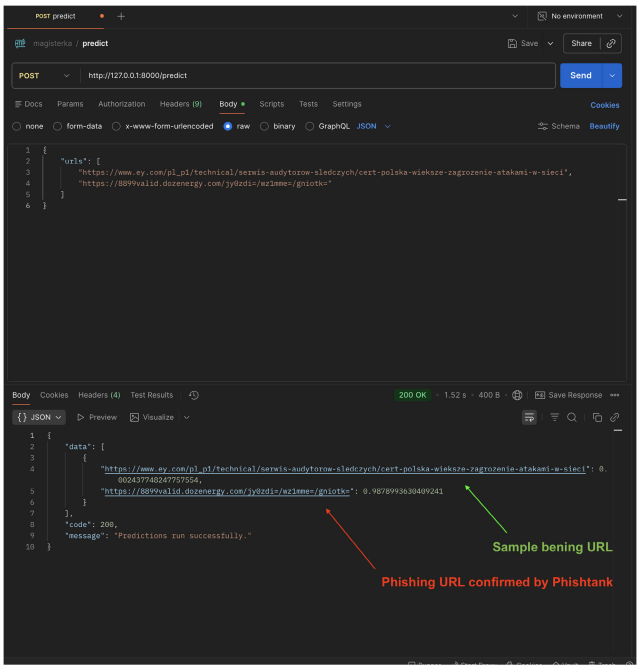


FIGURE 7. Local classification model API test with Postman.

To quantify uncertainty in these cross-dataset results, we repeated training and evaluation 10 times (fixed decision threshold 0.50) and report 95% bootstrap confidence intervals presented in Table 14. We additionally applied paired sign-flip permutation tests on F1 across repeats to assess whether

TABLE 6. URL overlap checks across datasets.

Key	C–G	C–G_T	C–P	G–G_T	G–P	G_T–P
raw_url	798 (1.07%)	180 (0.24%)	230 (0.49%)	2 (0.00%)	86 (0.18%)	13 (0.03%)
canonical_url	869 (1.19%)	205 (0.28%)	257 (0.55%)	3,118 (1.95%)	149 (0.32%)	29 (0.06%)
canonical_url_no_query	930 (1.40%)	253 (0.38%)	269 (0.57%)	7,413 (4.69%)	157 (0.33%)	32 (0.07%)
etld1_domain	2,046 (9.82%)	993 (4.77%)	531 (2.55%)	49,362 (45.56%)	2,297 (6.22%)	1,006 (2.73%)

Abbreviations: C = Custom, G = Grambeddings, G_T = Grambeddings_Test, P = PhiUSIIL.
Pairwise overlaps are reported as intersection count n and min-share $M = n/\min(|A|, |B|)$ (shown in %). Keys: raw URL, canonicalised URL (sorted query parameters), canonicalised URL without query parameters, and eTLD+1 domain (best-effort).

TABLE 7. Final selected features.

Feature	Type	Mean Imp.	CV	Reason
length_url	Lexical	0.0319	0.7859	Strong positive contributor
qty_hyphen_path	Lexical	0.0315	0.7164	Stable + important
has_brand_keyword	Security	0.0244	0.5643	Brand detection
subdomain_auth_count	Security	0.0204	0.4807	Auth signal
qty_nameservers	DNS	0.0084	0.3817	Infra signal
qty_dot_url	Lexical	0.0121	0.3866	URL structure
is_spf_domain	Security	0.0066	0.5139	SPF signal
time_domain_activation	Domain	0.0125	0.5328	Domain age

TABLE 8. Initial parameter space for Bayesian search cross validation.

Parameter	Range	Lower Bound Rationale	Upper Bound Rationale
`max_depth`	(3, 12)	Depth 3+ captures interactions	Depth above 12 overfits
`learning_rate`	(0.01, 0.3)	Below 0.01 is too slow	Above 0.3 unstable
`n_estimators`	(100, 500)	Below 100 weak	Above 500 diminishing returns
`subsample`	(0.6, 1.0)	Below 0.6 underfits	1.0 = full dataset
`colsample_bytree`	(0.6, 1.0)	60% $\approx$ 8–9 features	Full if optimal
`gamma`	(0, 5)	0 = unrestricted	> 5 over-prunes
`min_child_weight`	(1, 10)	1 = minimal reg.	> 10 too rigid

TABLE 9. Optimised XGBoost hyperparameters configuration.

Parameter	Initial Value	Final Value	Description	Engineering Reasoning
learning_rate	0.01	0.05	Conservative learning rate for stable convergence	Lower rates prevent overfitting to the training data and improve generalisation across different phishing datasets.
max_depth	12	8	Moderate tree depth for complex feature interactions	Deep enough to capture phishing patterns while shallow enough to prevent overfitting on unseen datasets.
n_estimators	500	300	Large ensemble for robust predictions	More trees provide better generalisation and stability across different data distributions.
scale_pos_weight	None	~1.954	Dynamic class weight (2.0× calculated weight)	Addresses phishing vs legitimate URL imbalance and achieves 80% recall vs 64% baseline.
gamma	0	0.1	Moderate minimum split loss requirement	Prevents overly complex trees while allowing beneficial splits for phishing detection.
reg_alpha	Default	0.1	Light L1 regularisation	Encourages sparse feature usage without overly penalising important phishing indicators.
reg_lambda	Default	0.1	Light L2 regularisation	Prevents overfitting while maintaining model flexibility for cross-dataset generalisation.
subsample	1.0	0.9	High row sampling with randomness	90% sampling provides variance reduction while maintaining ensemble diversity.
colsample_bytree	0.6	0.9	High feature sampling per tree	90% feature sampling balances model complexity with feature interaction capture.

observed differences are statistically distinguishable between models included in Table 15.

Recognising that success on one dataset does not guarantee optimal results in all scenarios is important.

TABLE 10. Model's cross-dataset phishing detection features.

Feature	Description	Calculation Method
length_url	Total character length of the URL	Count of all characters in complete URL
qty_hyphen_path	Number of hyphen characters in the URL path	Count of "-" in path
has_brand_keyword	Binary indicator of brand names in URL	Check predefined list
subdomain_auth_count	Count of authentication-related keywords	Count in subdomain
qty_nameservers	Number of nameserver records	DNS NS record count
qty_dot_url	Number of dot characters in the URL	Count of "." in URL
is_spf_domain	SPF record presence	DNS TXT lookup
time_domain_activation	Domain registration date (days since epoch)	WHOIS query

TABLE 11. Cross-dataset model performance comparison for generalisation test.

Dataset	Accuracy	Precision	Recall	Specificity	F1 Score	Threshold
Custom Test	90.72%	88.00%	94.56%	86.80%	91.16%	50%
GramBeddings Test	70.65%	67.31%	80.35%	60.94%	73.25%	85%
PhiUSIIL Test	72.16%	65.48%	73.76%	70.97%	69.37%	85%

TABLE 12. Baseline model comparison across datasets.

Dataset	Model	Accuracy	Precision	Recall	Specificity	F1	ROC-AUC
Custom Test	XGBoost	0.9072	0.8800	0.9456	0.8680	0.9116	0.9750
	LightGBM	0.9059	0.8769	0.9467	0.8640	0.9105	0.9750
	RandomForest	0.9032	0.9125	0.8944	0.9122	0.9034	0.9712
	CatBoost	0.8919	0.8570	0.9437	0.8389	0.8983	0.9680
	MLP	0.8952	0.8896	0.9050	0.8851	0.8972	0.9634
	LogisticRegression	0.7041	0.6929	0.7449	0.6622	0.7180	0.7814
GramBeddings Test	MLP	0.6881	0.6310	0.9065	0.4696	0.7441	0.7905
	RandomForest	0.6812	0.6275	0.8924	0.4699	0.7369	0.7578
	LightGBM	0.6463	0.5895	0.9641	0.3282	0.7317	0.7473
	CatBoost	0.6404	0.5856	0.9611	0.3195	0.7278	0.7646
	XGBoost	0.6423	0.5893	0.9403	0.3441	0.7245	0.7498
	LogisticRegression	0.6144	0.5812	0.8192	0.4093	0.6800	0.6344
PhiUSIIL Test	RandomForest	0.6382	0.5477	0.8812	0.4567	0.6755	0.7977
	XGBoost	0.6185	0.5319	0.8965	0.4110	0.6677	0.7996
	LightGBM	0.5958	0.5149	0.9430	0.3367	0.6661	0.7934
	CatBoost	0.5856	0.5082	0.9372	0.3231	0.6591	0.8210
	MLP	0.5977	0.5171	0.8911	0.3787	0.6544	0.8052
	LogisticRegression	0.4746	0.4471	0.9687	0.1058	0.6118	0.6401

Note: All models are evaluated at a fixed decision threshold of 0.50 to enable direct comparison of operating characteristics (precision/recall/specificity) across model families and datasets.

TABLE 13. Comparison of classification model results for phishing detection.

Model	Accuracy	Recall	Precision	F1 Score
Ours (Previous One Dataset Only)	97.80%	98.10%	97.30%	97.70%
Ours (Latest Cross-Dataset)	77.84%*	82.89%*	73.60%*	77.93%*
H. Lee et al. (2017)	99.29%	N/A	N/A	N/A
A. Jain et al. (2017)	99.09%	N/A	N/A	N/A
M. Zouina et al. (2018)	95.80%	N/A	N/A	N/A
J.S. Ambata et al. (2021)	99.53%	99.53%	99.53%	99.53%
A. Vazhayil et al. (2018)	99.92%	98.60%	98.80%	98.70%

* Average performance across three independent test datasets (Custom Test: 90.72%, GramBeddings Test: 70.65%, PhiUSIIL Test: 72.16%).

However, practical applications should involve monitoring and adjusting models to reflect current data and evolving conditions. That is why the latest study involves a cross-dataset comparison to find a balanced model for real-world applications.

IV. DISCUSSION

During our initial study, the XGBoost model proved to be the most effective, achieving the highest precision and accuracy in detecting malicious URLs. The tests showed that the tool can effectively identify threats and protect

TABLE 14. Cross-dataset performance of XGBoost with 95% bootstrap confidence intervals over 10 repeats (fixed threshold 0.50). Values are reported as mean [$CI_{2.5\%}$, $CI_{97.5\%}$].

Dataset	Accuracy	F1	ROC-AUC
Custom Test	0.9084 [0.9076, 0.9093]	0.9131 [0.9123, 0.9139]	0.9759 [0.9755, 0.9763]
GramBeddings Test	0.6468 [0.6440, 0.6498]	0.7292 [0.7266, 0.7318]	0.7580 [0.7549, 0.7610]
PhiUSIIL Test	0.6221 [0.6172, 0.6270]	0.6695 [0.6675, 0.6716]	0.8050 [0.8026, 0.8072]

CI details: percentile bootstrap CI over repeats (resamples=2000). Each repeat refits the model and evaluates on the same held-out test split for the given dataset.

TABLE 15. Paired sign-flip permutation tests across repeats for F_1 (two-sided; permutations = 20000; reference = XGBoost). ΔF_1 is computed as (model − XGBoost).

Dataset	Model	ΔF_1	p-value
Custom Test	LightGBM	−0.000983	0.0136
	RandomForest	−0.007408	0.0018
	CatBoost	−0.012365	0.0023
	MLP	−0.015809	0.0017
	LogisticRegression	−0.193753	0.0018
GramBeddings Test	MLP	+0.018289	0.0018
	RandomForest	+0.011746	0.0023
	LightGBM	+0.002546	0.1183
	CatBoost	−0.001806	0.2015
	LogisticRegression	−0.048617	0.0019
PhiUSIIL Test	RandomForest	+0.007359	0.0040
	LightGBM	−0.003627	0.0062
	CatBoost	−0.011745	0.0019
	MLP	−0.002931	0.6398
	LogisticRegression	−0.057107	0.0020

Note: Results are based on $n = 10$ paired repeats per dataset. p-values are uncorrected for multiple comparisons and are reported to support the robustness of cross-dataset differences requested by reviewers.

users from phishing attacks. The XGBoost model demonstrated exceptional performance, achieving 97.8% accuracy, 98.1% recall, and 97.3% precision using a classical train-test split on a single dataset. In the latest study, the model has been implemented for cross-dataset optimisation. The metrics may drop, but this is expected due to the extraction of dataset-specific features. The goal is to find the balance between the best metrics across different datasets.

The demo version of the plug-in has only been prepared for service X, although there are no technical obstacles to extending it to any website. Additionally, the application currently runs only as a Chrome extension, limiting its use to users of this specific browser. The server infrastructure presents another limitation: the application currently runs locally in a Docker container, as the Heroku cloud implementation has been discontinued; a new provider will be considered during the next stages of development. There is also no caching mechanism for the machine learning model's prediction algorithm, which could improve performance by reducing redundant computations. Moreover, the model is not optimised for concurrent requests, which can cause performance bottlenecks under high load.

Despite these technical and practical limitations, security remains a critical concern. Adversaries commonly obfuscate or manipulate URL signals (e.g., shorteners, homoglyphs, excessive subdomains, per cent-encoding) to hide features used by static detectors. While our cross-dataset, multi-signal feature selection reduces reliance on any single attribute, practical deployments should combine periodic retraining with fresh adversarial samples, runtime dynamic checks (such as landing-page rendering and WHOIS/DNS), or ensemble defences to maintain robustness.

XGBoost models often have a large memory footprint, which can limit their use in web browsers. For future development, we may need to address this by pruning trees, reducing the ensemble size, exporting models via Treelite or ONNX with post-training quantisation (e.g., int8), or distilling the ensemble into a smaller predictor. These techniques typically substantially reduce runtime and storage requirements. They incur only modest accuracy loss and can be readily integrated into the plug-in build pipeline [23].

Taken together, these insights guide our future steps and mark this study as a prequel to future model improvements. Further work on the plug-in will proceed in two ways: first, by developing a method to automatically update the model, and second, by combining multiple models—including XGBoost, deep learning algorithms, and classical linear regression—to find the best phishing detection solution.

V. CONCLUSION

XGBoost offers several practical advantages for phishing URL detection. It supports efficient training, achieves strong predictive performance with structured features, and provides feature-importance measures that aid model interpretation

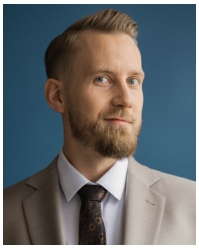
and feature engineering. These properties make XGBoost well-suited for real-time applications, such as browser plug-ins, where low latency and explainability are important.

At the same time, our cross-dataset experiments reveal clear limitations. Model performance decreases under dataset shift, which reflects biases in the data and differences in data collection methods. In addition, attackers continuously adapt their obfuscation techniques, and runtime or resource constraints (e.g., model size in browser-based settings) can limit deployment options. As a result, real-world systems require continuous monitoring, regular retraining with updated data (including adversarial examples), and additional safeguards, such as ensemble methods or runtime validation checks, to maintain robustness.

In summary, XGBoost provides a practical and interpretable baseline for phishing detection. However, its effective deployment in production systems depends on ongoing dataset curation, model maintenance, and engineering strategies that address adversarial behaviour and distribution shifts. Future work will focus on these operational challenges, as well as on model compression and lightweight deployment techniques.

REFERENCES

[1] A. Aleroud and L. Zhou, "Phishing environments, techniques, and countermeasures: A survey," *Comput. Secur.*, vol. 68, pp. 160–196, Jul. 2017.
[2] T. Manyumwa, P. F. Chapita, H. Wu, and S. Ji, "Towards fighting cybercrime: Malicious URL attack type detection using multiclass classification," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Mali, Dec. 2020, pp. 1813–1822.
[3] W. J. Hajara, M. Gul, A. Zafar, U. A. Yasin, and A. U. Waqas, "A comparative analysis of phishing website detection using XGBoost algorithm," *J. Theor. Appl. Inf. Technol.*, vol. 97, no. 5, pp. 1434–1443, 2019.
[4] S. N. N. V. Kumaraswamy and M. W. Kumar, "Comparative analysis of machine learning algorithms for phishing website detection," *Int. J. Sci. Res. Arch.*, vol. 12, no. 1, pp. 293–298, May 2024.
[5] A. A. Zuraiq and M. Alkasassbeh, "Review: Phishing detection approaches," in *Proc. 2nd Int. Conf. New Trends Comput. Sci. (ICTCS)*, Amman, Jordan, Oct. 2019, pp. 1–6.
[6] S. Patil and S. Dhage, "A methodical overview on phishing detection along with an organized way to construct an anti-phishing framework," in *Proc. 5th Int. Conf. Adv. Comput. Commun. Syst. (ICACCS)*, Coimbatore, India, Mar. 2019, pp. 588–593.
[7] E. S. Aung, C. T. Zan, and H. Yamana, "A survey of URL-based phishing detection," in *Proc. 11th Forum Data Eng. Inf. Manage. (DEIM Forum)*, vol. G2-3. Tokyo, Japan: Department of Computer Science andCommunication Engineering, Graduate School of Fundamental Science and Engineering, Waseda University, 2019.
[8] A. U. Z. Asif, H. Shirazi, and I. Ray, "Machine learning-based phishing detection using URL features: A comprehensive review," in *Stabilization, Safety, and Security of Distributed Systems*, 2023, pp. 481–497.
[9] A. S. Bozkir, F. C. Dalgic, and M. Aydos, "GramBeddings: A new neural network for URL based identification of phishing web pages through N-gram embeddings," *Comput. Secur.*, vol. 124, Jan. 2023, Art. no. 102964.
[10] O. A. Alzubi, "BotNet attack detection using MALO-based XGBoost model in IoT environment," in *Proc. 3rd Int. Conf. Comput. Commun. Netw.* Singapore: Springer, 2025, pp. 679–690, doi: 10.1007/978-981-97-2671-4_50.
[11] D. M. Divakaran and A. Oest, "Phishing detection leveraging machine learning and deep learning: A review," *IEEE Secur. Privacy*, vol. 20, no. 5, pp. 86–95, Sep. 2022.
[12] A. Ozcan, C. Catal, E. Donmez, and B. Senturk, "A hybrid DNN–LSTM model for detecting phishing URLs," *Neural Comput. Appl.*, vol. 35, no. 7, pp. 4957–4973, Mar. 2023.
[13] H. Nasser, F. Trad, and A. Chehab, "Stacking large language models is all you need: A case study on phishing url detection," *J. Artif. Intell. Soft Comput. Res.*, vol. 15, no. 4, pp. 337–356, Oct. 2025.
[14] F. Rashid, B. Doyle, S. C. Han, and S. Seneviratne, "Phishing URL detection generalisation using unsupervised domain adaptation," *Comput. Netw.*, vol. 245, May 2024, Art. no. 110398.
[15] Ö. Ş. Akçam, A. Tekerek, and M. Tekerek, "Development of BiLSTM deep learning model to detect URL-based phishing attacks," *Comput. Electr. Eng.*, vol. 123, Apr. 2025, Art. no. 110212.
[16] M. Misiek and T. Hyla, "Preventing phishing attacks with browser-based URL detection," in *Proc. 58th Hawaii Int. Conf. Syst. Sci. (HICSS)*, Honolulu, USA, 2025, pp. 7309–7316, 2025, doi: 10.24251/HICSS.2025.874.
[17] Cisco Talos Intelligence Group. *Phishtank*. Accessed: Mar. 2024. [Online]. Available: https://phishtank.com/
[18] Ada. *Ada URL Dataset*. Accessed: 2024. [Online]. Available: https://github.com/ada-url/url-dataset/blob/main/README.md
[19] H. Mustafavi. *Kaggle Phishing URLs*. Accessed: 2025. [Online]. Available: https://www.kaggle.com/datasets/hassaanmustafavi/phishing-urls-dataset
[20] ESDAUNG. *Phishdataset*. Accessed: 2025. [Online]. Available: https://github.com/ESDAUNG/PhishDataset
[21] A. S. Bozkir, M. A. Dalgiç, and F. C. Dalgiç. *Grambeddings Dataset*. Accessed: 2025. [Online]. Available: https://web.cs.hacettepe.edu.tr/~selman/grambeddings-dataset/
[22] A. P. and S. Chandra. (2024). *Phiusiil Phishing URL Dataset*. [Online]. Available: https://archive.ics.uci.edu/dataset/967/phiusiil+phishing+url+dataset
[23] J. Brownlee. *XGBoosting. Making You Awesome at XGBoost*. Accessed: 2024. [Online]. Available: https://xgboosting.com/xgboost-use-less-memory/



MILOSZ MISIEK received the M.Sc. degree in computer science from West Pomeranian University of Technology, Szczecin, Poland, in 2024, where he is currently pursuing the Ph.D. degree in computer science.

He is a Software Engineer at a product-based company, focusing on enhancing web application functionality. He has experience in full-stack development and his professional work combines academic research with practical software engineering. His research interests include the application of artificial intelligence in cybersecurity and modern web technologies.

TOMASZ HYLA received the M.Sc. degree in computer science from Szczecin University of Technology, in 2007, and the Ph.D. degree in computer science and cryptography from West Pomeranian University of Technology, Szczecin, Poland, in 2011. Since 2020, he has been an Associate Professor and the Head of the Information Security Research Group, West Pomeranian University of Technology. He works in the cybersecurity area. His primary research interests include blockchain technology and the application of AI in cybersecurity.